Project Report: Concurrent Odds-and-Evens Game in Go

Eduard Toni Alexandru

September 20, 2026

1 Analysis

The goal of this project is to design and implement an Odds-and-Evens game in Go, played by N players, structured as a knockout tournament. The requirements and constraints identified during the analysis phase are outlined below:

- **Tournament Structure:** The total number of players N must be a power of 2 ($N = 2^m$), allowing the game to be organized cleanly into $m = \log_2(N)$ rounds. At each round, matches run concurrently, and winners advance to the subsequent round until a final match determines the overall champion.
- **Concurrency Model (Message Passing Only):** A strict architectural constraint is enforced: *no shared memory is allowed.* All communication, coordination, and synchronization between concurrent entities must occur exclusively via message passing using Go channels.
- **Game Mechanics and Tie-Handling:** In each match, two players independently choose a move (0 or 1). A coordinator (or match referee) also picks a move. If the two players pick the same move, a tie occurs, and the match must be replayed until a definitive winner is established.
- **Input Validation:** The application must validate user input to ensure that N is strictly a positive integer and a valid power of 2.

2 Design

To fulfill the requirements without shared memory, a **Centralized Peer-to-Peer (Coordinator-Worker)** architecture has been chosen. In this model, active concurrent entities coordinate the entire workflow.

2.1 Active Elements at Play

1. **The Main Process (`main`):** Acts as the system bootstrapper. It collects and validates user input ($N = 2^m$) and kicks off the tournament by initializing the central coordinator.
2. **The Coordinator (`Coord`):** Manages the global state of the tournament across rounds. It maintains the list of active players, computes pairings for each round, spawns concurrent match workers, and aggregates results via channels.
3. **Match Workers (`playMatch`):** Spawned concurrently as goroutines for each match in a round. Each match worker acts as an independent localized coordinator that interacts with the two competing player goroutines.

4. **Players (`Player`):** Modeled as independent goroutines that generate and transmit their random moves via dedicated channels upon request.

2.2 General Workflow

1. **Initialization:** The system starts, validates N, and creates the initial pool of N players, assigning each a unique identifier.

2. **Round Execution:** For each round r, the coordinator divides the active player slice into pairs. For each pair, it spawns a concurrent ‘playMatch‘ goroutine and passes a shared result channel (‘resultChan‘).

3. **Match Resolution & Tie-Breaks:** Inside ‘playMatch‘, player goroutines are triggered. They send their moves through private channels. If a tie occurs ($m_1 == m_2$), the match loops and re-attempts. Once a winner is decided, ‘playMatch‘ sends the winner's info back to the coordinator and terminates.

4. **Progression:** The coordinator collects $N/2$ results from the result channel, builds the next round's player roster, and repeats the process until only one player remains.

3 Implementation & Complexity Analysis

3.1 Pseudocode Structure

The core logic implemented in Go follows this structured procedure:

Listing 1: System Pseudocode

```
// Main entry point with input validation
function main()
    repeat
        read nPlayers
        if nPlayers <= 0 or (nPlayers & (nPlayers - 1)) != 0 then
            print "Error: must be a power of 2"
            continue
        end if
        break
    end repeat
    Coord(nPlayers)
end function

// Coordinator managing tournament rounds
function Coord(nPlayers)
    players = array of N players initialized with IDs 1 to N
    roundNum = 1

    while length(players) > 1 do
        nMatches = length(players) / 2
        resultChan = channel of size nMatches

        for i = 0 to length(players) step 2 do
            matchIdx = i / 2
            spawn playMatch(matchIdx, players[i], players[i+1], roundNum
                , resultChan)
        end for

        nextRoundPlayers = empty array of size nMatches
        for j = 0 to nMatches - 1 do
            res = receive from resultChan
            nextRoundPlayers[res.matchIdx] = res.winner
        end for

        players = nextRoundPlayers
        roundNum = roundNum + 1
    end while
    print "Champion is players[0]"
end function

// Match worker handling a single pairing and ties
function playMatch(matchIdx, p1, p2, roundNum, resultChan)
    loop
        coordMove = random(0 or 1)
        spawn Player(p1.id, moveChan1)
        spawn Player(p2.id, moveChan2)

        m1 = receive from moveChan1
        m2 = receive from moveChan2

        if m1 == m2 then
```

```
            continue // Tie: replay match
        end if

        if m1 == coordMove then
            winner = p1
        else
            winner = p2
        end if

        resultChan <- MatchResult(matchIdx, winner)
        break
    end loop
end function
```

3.2 Message Complexity Analysis

To evaluate communication overhead, we analyze the number of messages exchanged via channels across the tournament:

- **Total Matches:** In a knockout tournament with N players, the total number of matches played across all rounds is:

$$\sum_{k=1}^{\log_2 N} \frac{N}{2^k} = \frac{N}{2} + \frac{N}{4} + \cdots + 1 = N - 1$$

- **Messages per Match (No Ties):** Each match involves 2 player move messages sent to the match worker, plus 1 result message sent from the match worker back to the coordinator. This equals 3 base messages per match.
- **Impact of Ties:** Since players pick moves randomly (0 or 1), the probability of a tie on any given attempt is $P(\text{tie}) = \frac{1}{2}$. The expected number of attempts per match follows a geometric distribution with mean $\mathbb{E}[\text{attempts}] = \frac{1}{1-0.5} = 2$.
- **Total Expected Messages:** Accounting for tie replays, each match requires an average of 2 attempts. Thus, the expected number of player move messages per match is $2 \times 2 = 4$, plus 1 result message, totaling 5 messages per match.
- **Total System Message Complexity:**

$$\text{Total Messages} = 5 \times (N - 1) = \mathcal{O}(N)$$

This demonstrates that the system achieves high concurrency and efficiency with a linear message complexity relative to the number of players.

3.3 Message Complexity Comparison: Deterministic Assignment

To rigorously evaluate our project, we can contrast the message complexity of our randomized tie-handling implementation with a hypothetical deterministic variant:

1. **Deterministic Version (Index-Based Assignment):**
    - By assigning moves based on player indices (e.g., even = 0, odd = 1), ties are completely eliminated, requiring exactly 1 attempt per match.
    - Each match requires 2 player move messages and 1 result message (totaling 3 messages per match).

- **Total Messages:** $3 \times (N-1) = \mathcal{O}(N)$.

Conclusion: Both versions maintain a linear $\mathcal{O}(N)$ **message complexity** characteristic of centralized coordination. While the deterministic version reduces the constant multiplier by removing tie-replays, the randomized version correctly preserves the true game rules and independent peer behaviors required by the assignment specification.